TECHNISCHE UNIVERSITÄT WIEN
Vienna | Austria

# GPU-Accelerated Detection and Classification of Jigsaw Puzzle Pieces

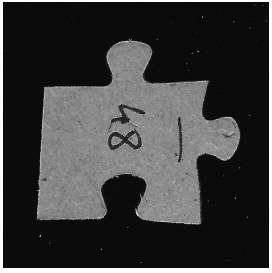
## Serial C++ Baseline and Performance-Optimized CUDA Pipeline

Jakob Olsacher  Ananthalakshmi Vinod Iyer
Philipp Sepin  Jan-Hill Arganda  Tobias Ponesch

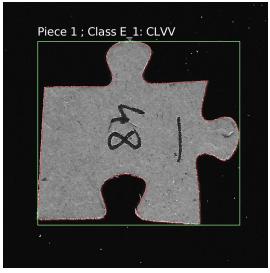
GPU Architecture & Computing, TU Wien

Summer Semester 2026

# Project Goal



Input image




Classified output

## Objective

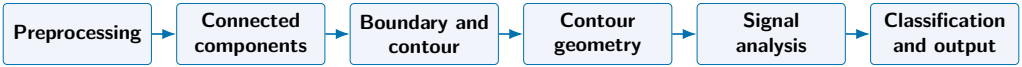
Detect every non-overlapping puzzle piece and assign a rotation-normalized class from its four edge types.

- **L**: straight edge
- **V**: knob
- **C**: hole
- Example: `CLVV`

## Central Question

Which parts of the pipeline benefit from GPU execution, and where do setup, synchronization, or data transfer dominate?

# One Algorithm, Two Implementations




**SERIAL** `pipeline.cpp`

C++ correctness and timing baseline using loops and standard containers.

**PARALLEL** `pipeline.cu`

Same stages and outputs, but reorganized with CUDA kernels, batched buffers, and selected CPU stages where sequential logic remains cheaper.

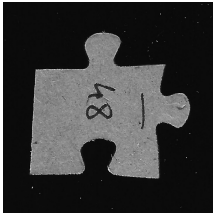
## CUDA Optimization Lens

A GPU stage only pays off when useful parallel work outweighs setup, transfer, launch, and synchronization overhead.

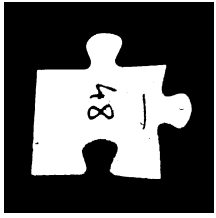
# Image Preprocessing: Serial

**SERIAL**

1. RGB to grayscale
2. Min/max normalization to $[0, 255]$
3. Gaussian blur with clamped borders
4. 256-bin histogram and Otsu threshold
5. Binary thresholding
6. Morphological opening: erosion then dilation



before



after

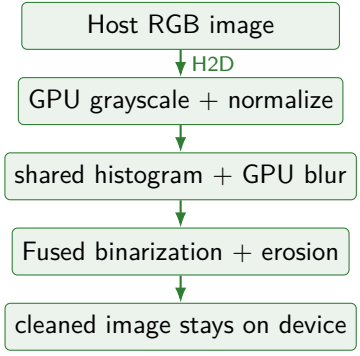
## Serial Cost Driver

Gaussian blur and morphology repeatedly read neighborhoods for every image pixel.

# Image Preprocessing: Parallel

**PARALLEL**

- Grayscale conversion, normalization, and Gaussian blur use straightforward pixel-level GPU parallelization.
- Gaussian weights are cached in CUDA constant memory.
- Blocks build shared-memory histograms before the global merge.
- Binarization and erosion are fused into one tiled kernel.
- Morphology stages neighborhood pixels in shared memory.



# Optimization Deep Dive: Fused and Tiled Morphology

## Before

Separate full-image passes repeatedly read the same neighborhoods for morphological filtering.

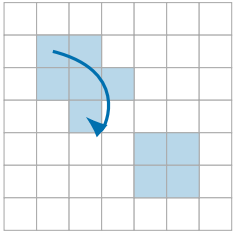
## Current CUDA Implementation

- Fuse binarization with erosion to reduce global memory reads
- Load **tiles plus halo** into shared memory

# Connected Components: Serial

**SERIAL**

1. Scan the binary image from top-left to bottom-right.
2. Start a queue-based breadth-first search at every new foreground component.
3. Traverse the eight-neighborhood while updating area and bounding-box extrema.
4. Discard regions below `min_area`.



Queue-based flood fill follows one component at a time

## Output

A label image plus one `Region` record per retained puzzle piece.

## Dependency

Each queue frontier depends on the preceding traversal step.

# Connected Components: Parallel

**PARALLEL**

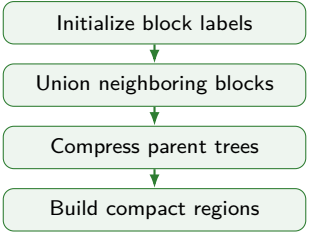
1. Treat each $2 \times 2$ block containing foreground as one initial unit.
2. Merge neighboring block labels with GPU union–find.
3. Compress parent trees and emit final per-pixel labels.
4. Build region statistics on the GPU; copy compact regions only.

## Key Performance Optimization

BUF-inspired removes the convergence loop, repeated full-image launches, and host-visible `changed` flag.

## Why $2 \times 2$ Blocks?

With 8-connectivity, foreground pixels inside a $2 \times 2$ block are already connected. Fewer labels mean less union–find and memory work.



# Optimization Deep Dive: Why BUF Replaced Propagation

## Old Bottleneck

Label propagation required many full-image iterations for large pieces.

## Profiler-Driven Decision

Profiling exposed CCL as launch- and synchronization-heavy.

## BUF-Inspired Replacement

1. Initialize one label per $2 \times 2$ block.
2. Merge equivalent neighboring blocks.
3. Compress union–find trees.
4. Expand roots to pixel labels.
5. Build region statistics on the GPU.

## Expected Effect

Fewer labels and memory accesses.

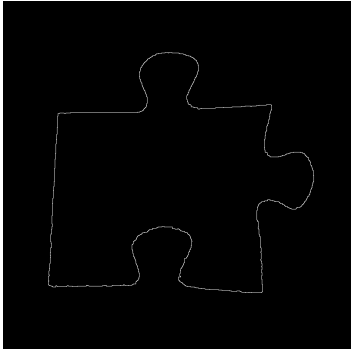
# Contour Extraction: Serial

**SERIAL**

1. Starting from the first nonzero pixel.
2. Walking around the contour one step at a time.
3. From neighbour to neighbour.
4. Then simplify contour by dropping points along lines.

**Moore Neighbour Tracing**

An inherently sequential algorithm.



Extracted one-pixel boundary

# Contour Extraction: Parallel

**HYBRID**

## Why not fully parallel?

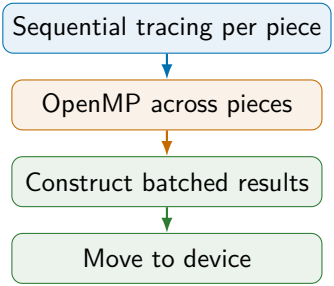
Parallel alternatives return an unordered set of contour points. Ordering them is highly complex.

## OpenMP across Pieces

Pieces are processed independently using OpenMP.

## Flat Batch Layout for next Stages

Results are returned in a flat batched structure.



# Contour Smoothing and Radial Signal: Serial

**SERIAL**

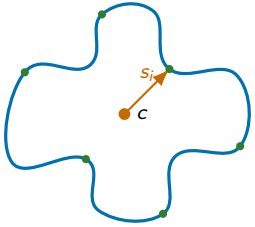
1. Smoothing the contour with a moving average.
2. Estimating the center of an enclosing circle from coordinate extrema.

$$c_x = \frac{x_{\min} + x_{\max}}{2}, \qquad c_y = \frac{y_{\min} + y_{\max}}{2}$$

3. Computing the distance of each contour point to this center.

$$s_i = \sqrt{(x_i - c_x)^2 + (y_i - c_y)^2}$$

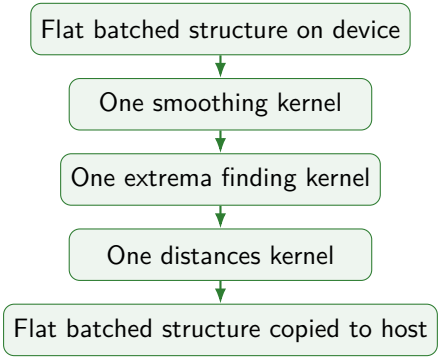
4. Thereby producing the radial signal.



# Contour Smoothing and Radial Signal: Parallel

**PARALLEL**

- One batched kernel smooths the contours of all pieces.
- One kernel for finding all coordinate extrema.
- Even faster than Thrust, as all four extrema are computed in one kernel.
- One kernel to compute the distances fully parallel.



# Optimization Deep Dive: Thrust vs Custom Kernel

## Why Thrust was slow

- Highly optimized parallel reductions.
- But needed four separate sequential calls, two maxima, two minima.
- High overhead.

## Why Custom Kernel is faster

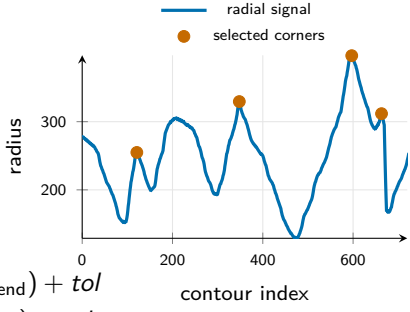
- Finds all four extrema in a single kernel launch.
- And does this for all pieces in a single launch.
- Uses shared memory tree reductions.
- But could have used warp shuffles.

# Signal Analysis and Classification: Serial

**SERIAL**

1. Smooth the radial signal and peak detection.
2. Filter candidates by prominence, sharpness, and distance.
3. Greedily select four well-separated corners.
4. For each edge, compare extrema against its two shoulders:

$$\text{Classification} = \begin{cases} \text{Knob (V)} & \text{if } s_{\max} > \max(v_{\text{start}}, v_{\text{end}}) + tol \\ \text{Hole (C)} & \text{if } s_{\min} < \min(v_{\text{start}}, v_{\text{end}}) - tol \\ \text{Flat (L)} & \text{otherwise} \end{cases}$$




5. Final classification through lookup table. e.g. 'VVCL'

# Signal Analysis and Classification: Parallel

**HYBRID**

## Parallelized

- Signal smoothing: Batched over all piece signals
- Peak detection: Batched local-maximum mask kernel

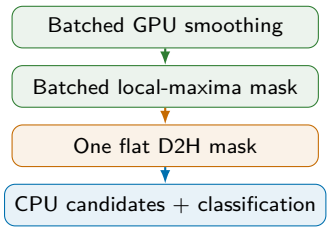
## Kept on the Host

Operations on the resulting small candidate sets:

- Prominence filtering & Four-corner selection
- Edge classification & Lookup.

## Reason for the Split

Batching removes per-piece launch/copy overhead.



# Optimization Deep Dive: Smoothing & Local-Maxima Mask

## Grid pattern for both kernels

- `blockIdx.y` = piece, `blockIdx.x*blockDim.x+threadIdx.x` = local sample.

## Signal Smoothing

- Centered averaging window of width $k$.
- Each thread sums $k$ neighbors independently.

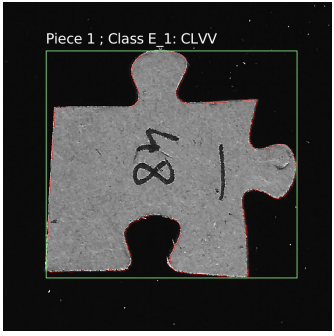
## Local-Maxima Mask

- Asymmetric test: `val >= left && val > right`.
- Writes a binary mask for candidate analysis happens which happens on CPU.

# Pipeline Integration and Visualization: Serial

**SERIAL**

- `run()` executes all stages in a fixed order.
- Normal CPU containers keep the reference path easy to inspect.
- Detected regions are processed one by one.
- Timings exclude image loading and final file writing.
- Host-side visualization draws contours, boxes, and labels.




Final host-rendered overlay

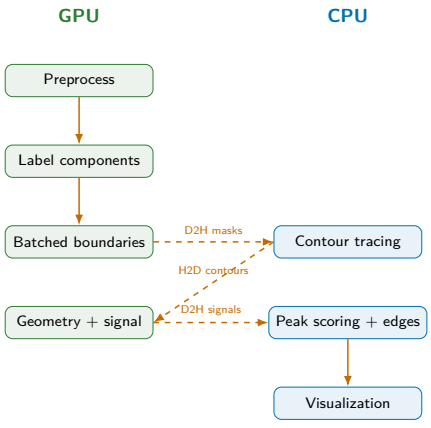
**Reference Role**

Correctness baseline and timing reference for the CUDA pipeline.

# Pipeline Integration and Visualization: Parallel

**HYBRID**

- Pipeline owns device buffers and reusable scratch.
- Cleaned mask and compact labels stay on the GPU.
- Region and contour buffers are allocated once sizes are known.
- Contours and signals use flat batched buffers.
- Sequential or tiny stages stay CPU-side.




## Key Decision

Move data only where the next stage needs it; subfunctions write into pipeline-owned buffers instead of allocating and returning data.

# Optimization Deep Dive: Pipeline Cleanup Impact

## What Changed

- Batch work across all pieces.
- Reuse pipeline-owned device buffers.
- Charge allocations to `cuda_setup`.
- Replace per-piece copies with flat batched transfers.

## Why It Matters

The pipeline stopped paying allocation, copy, launch, and sync overhead for every small piece of work.

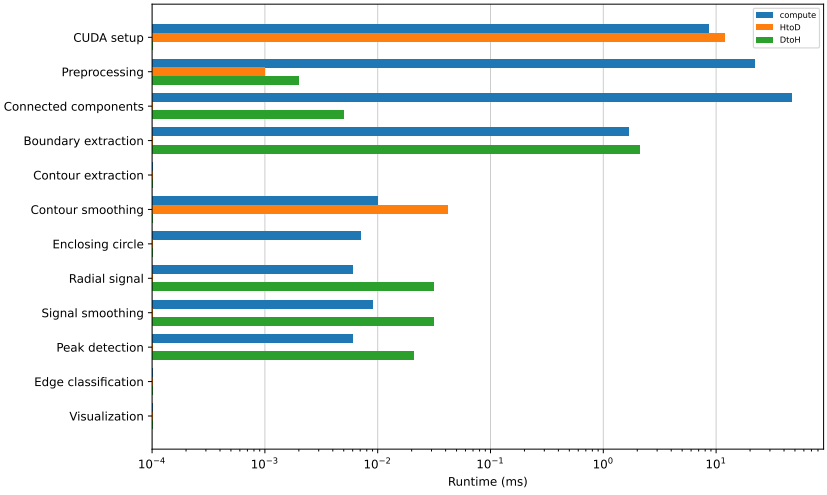
## Measured Effect

| CUDA activity | Old | New |
|---|---|---|
| Runtime API calls | 6330 | 166 |
| Kernel launches | 1318 | 52 |
| Memory-copy calls | 1084 | 13 |
| Synchronizations | 2062 | 39 |

## Takeaway

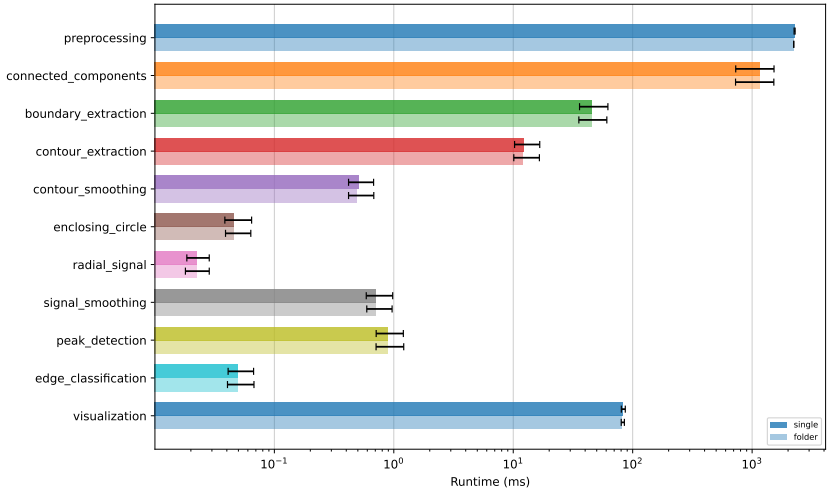
The speedup comes from pipeline structure as much as from individual kernels.

# CUDA Compute and Transfer Overhead by Pipeline Stage



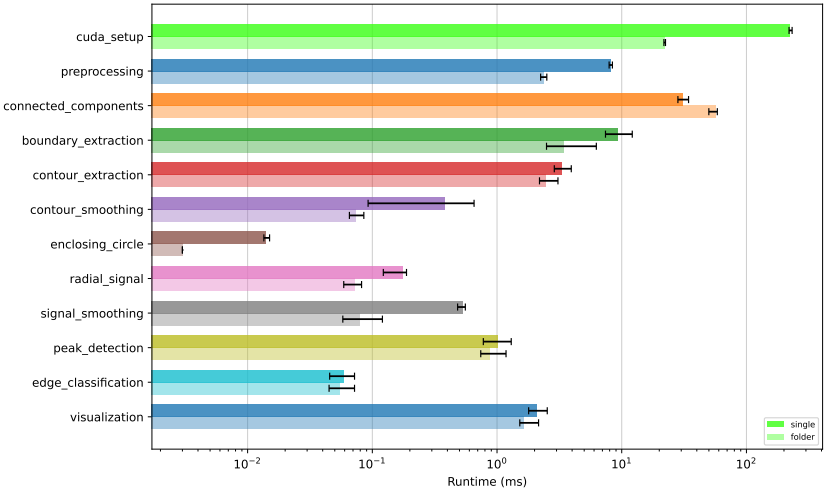

Data transfers are concentrated in CUDA setup and boundary extraction, while preprocessing and connected-component labeling are dominated by GPU computation.

# Serial Pipeline: Single-Image vs. Folder Mode



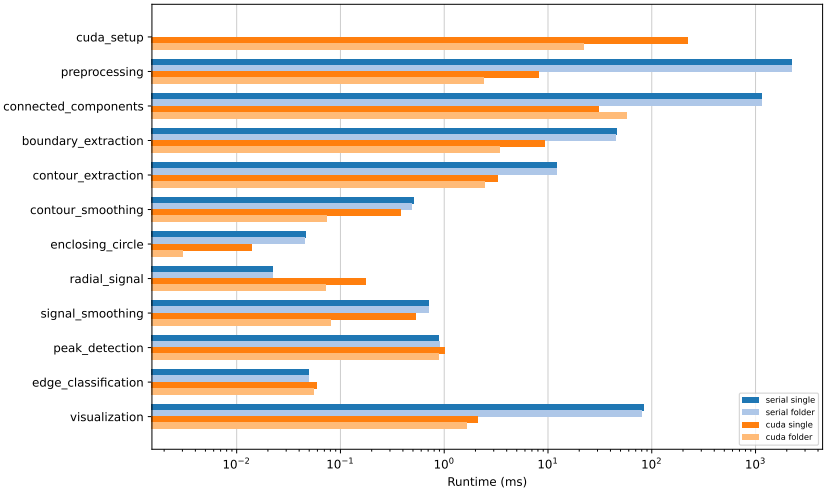

Per-stage runtimes are similar in both modes because the serial pipeline has no one-time CUDA initialization cost.

# CUDA Pipeline: Single-Image vs. Folder Mode



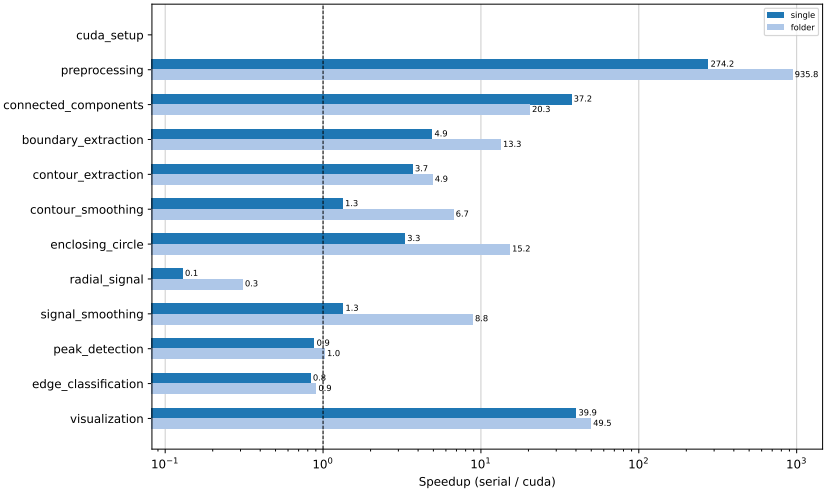

Folder mode substantially reduces the per-image CUDA setup time by reusing the CUDA context across multiple images.

# Serial and CUDA Stage Runtime Comparison



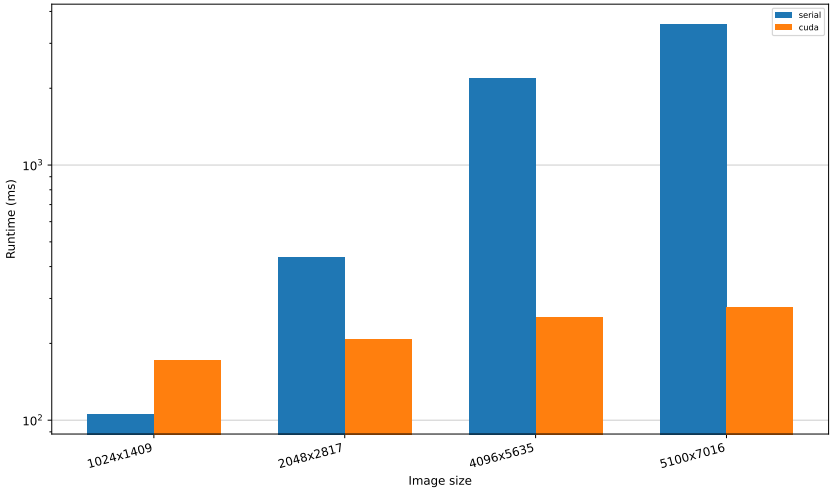

CUDA strongly reduces the dominant processing stages, while folder mode additionally amortizes the CUDA setup cost.

# Per-Stage CUDA Speedup



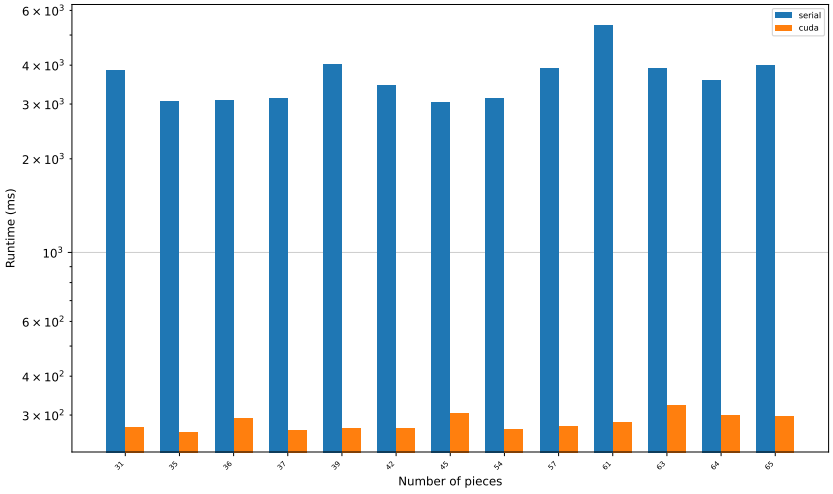

Preprocessing and connected-component labeling achieve the largest speedups, whereas several small per-piece stages remain near or below break-even.

# End-to-End Runtime by Image Resolution



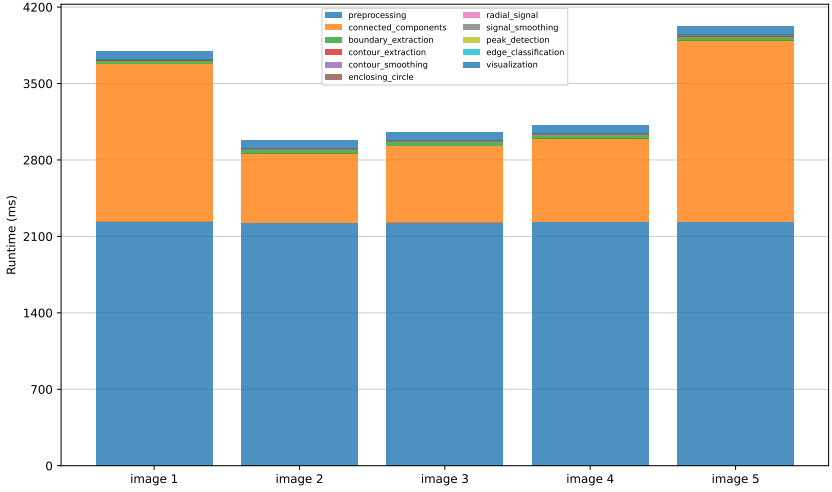

Serial runtime increases strongly with image resolution, whereas CUDA runtime grows only moderately. CUDA becomes faster once its setup overhead is amortized by a larger image.

# End-to-End Runtime by Puzzle Pieces



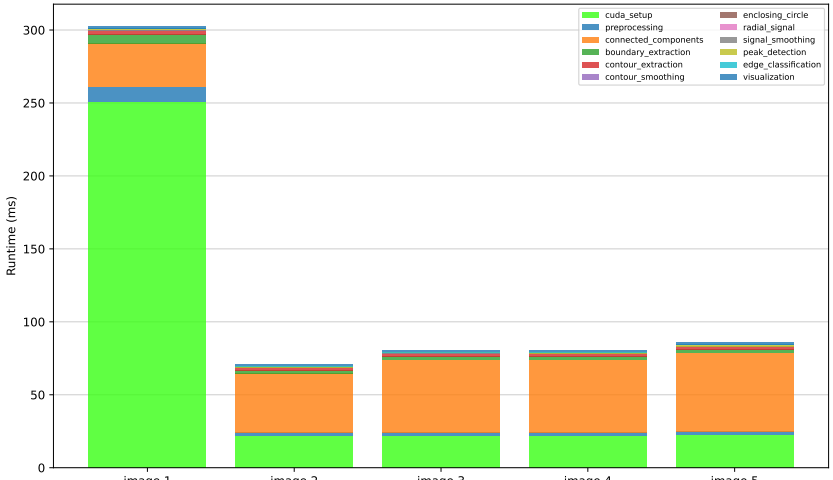

Runtime does not increase monotonically with the number of pieces. Variations are primarily caused by connected-component labeling and the topology of the segmented foreground.

# Serial Runtime Breakdown Across Images




Runtime differences between images are primarily caused by connected-component labeling, while preprocessing remains nearly constant.

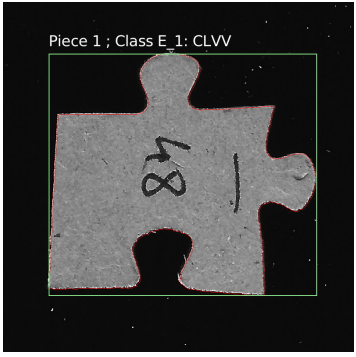
# CUDA Runtime Breakdown Across Images




Connected-component labeling is the dominant processing stage and varies with the topology and complexity of the segmented regions.

# Conclusion

- CUDA reaches 12.1× speedup on the largest single image.
- Folder processing reaches 37.3× by amortizing CUDA setup.
- Fusion, BUF, batching, and scratch reuse drive the gains.
- Small inputs and tiny irregular stages remain CPU-friendly.
- Current folder mode still recreates buffers per image, so persistent buffers and streaming remain future work.




**Takeaway**

GPU performance depends on granularity: batch enough work to make initialization and transfers negligible.

# Learnings

## Technical Learnings

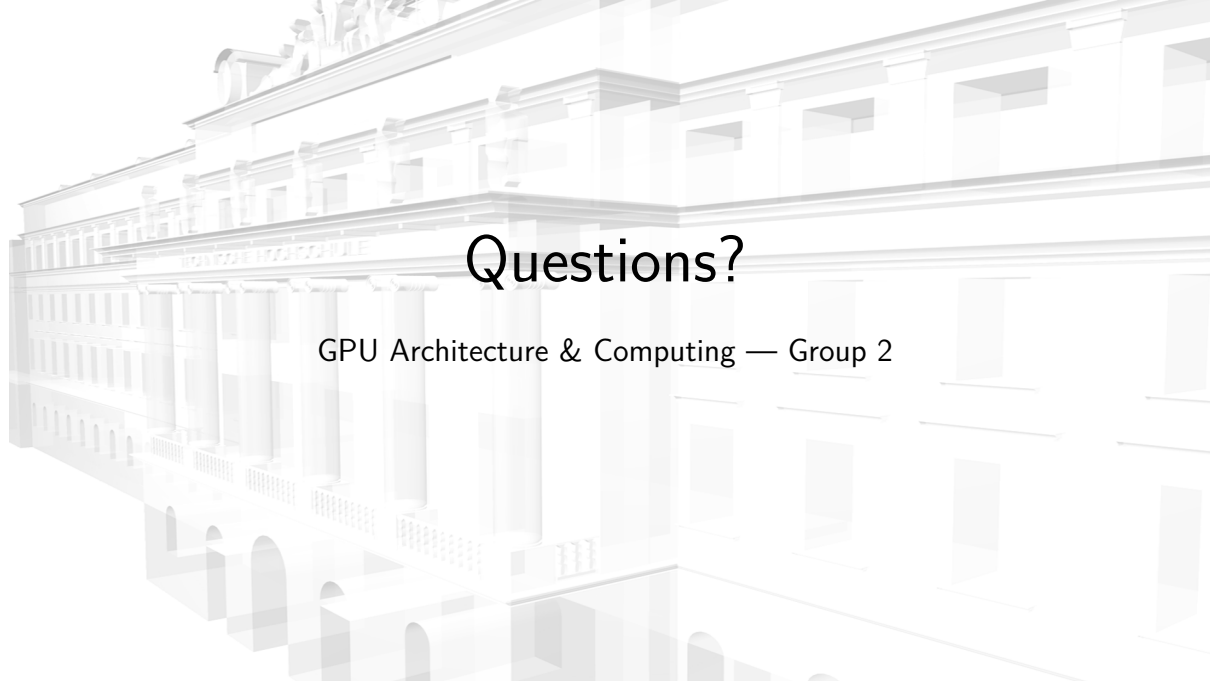
- GPU stages need enough independent work.
- Small, irregular, and branch-heavy stages can still be faster on the CPU.
- Batching is essential to amortize CUDA setup, transfers, and kernel launches.

## Project Learnings

- Integrating independently developed stages is difficult and an iterative process.
- Consistent data ownership avoids temporary copies and allocation decisions later.
- Performance optimization requires profiling the full pipeline, not only isolated kernels.

## Takeaway

CUDA helped most when the pipeline batched work and kept data movement controlled.



# Questions?

GPU Architecture & Computing — Group 2

# Benchmark Configuration

## Data Set

- 13 manually counted puzzle images
- Between 31 and 65 pieces per image
- Same images at four resolutions
- Five runs per image and implementation
- 260 single-image measurements per implementation

| Resolution | Pixels |
|---|---|
| $1024 \times 1409$ | 1.44 M |
| $2048 \times 2817$ | 5.77 M |
| $4096 \times 5635$ | 23.08 M |
| $5100 \times 7016$ | 35.78 M |

## Aggregation

Median over five runs per image, followed by the median over all 13 images of one resolution.

# What Is Included in the Timings?

## Included

- CUDA initialization
- Device allocation and input upload
- All processing stages
- CPU-side hybrid stages
- Overlay rendering

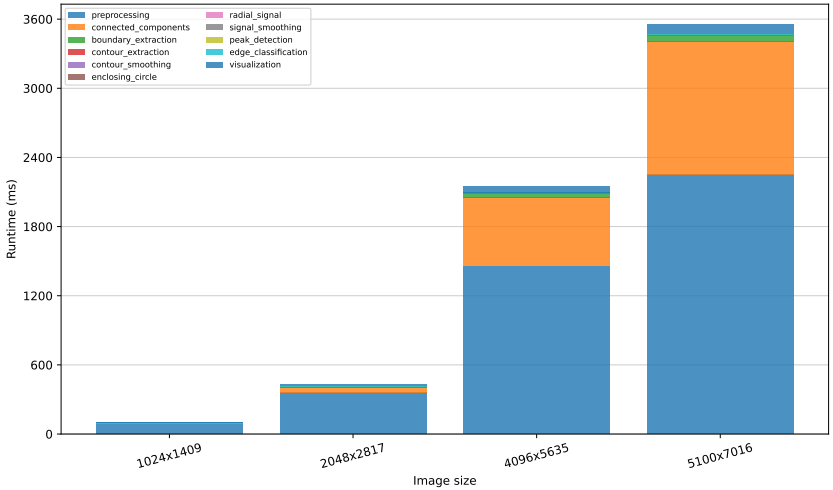
## Excluded

- Loading the input image
- Writing the overlay image
- Build and compilation time
- Benchmark-analysis scripts

## Timing Interpretation

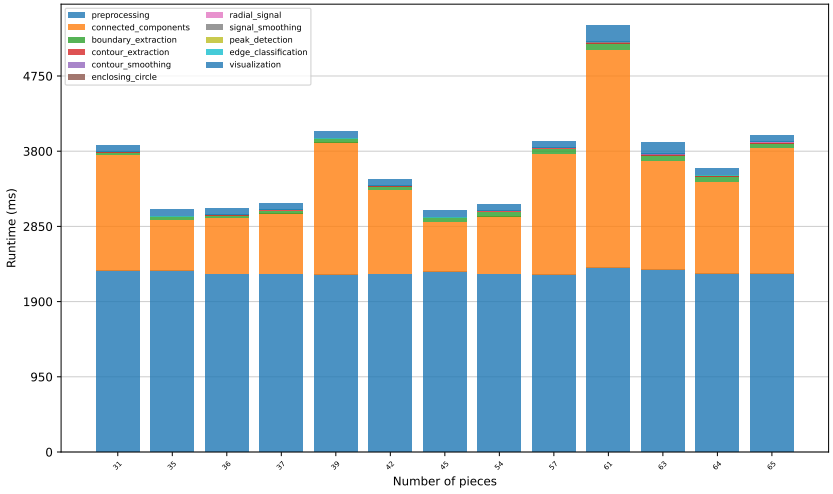
The CSV files contain host-side wall-clock timings. CUDA stages include synchronization and transfer effects and are not pure kernel timings. `cuda_setup` includes initialization, allocation, and input upload.

# Serial Runtime Scaling with Image Resolution



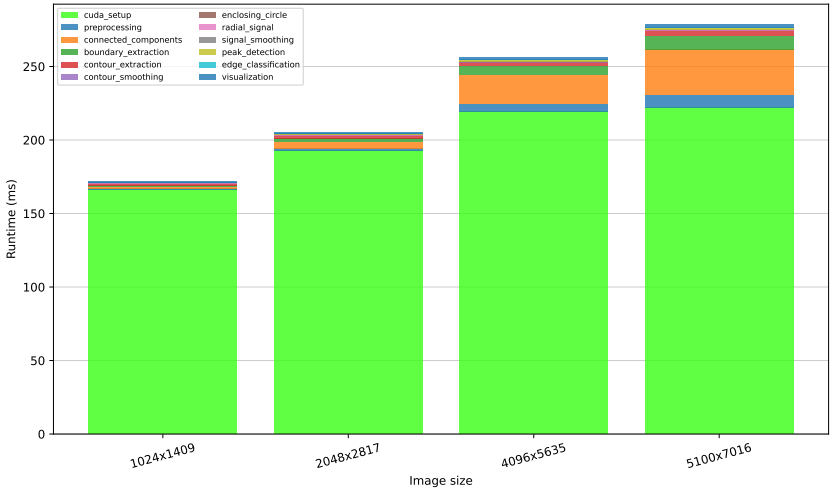

Serial runtime increases strongly with image resolution. Preprocessing and connected-component labeling account for most of the execution time.

# Serial Runtime Across Puzzle Images



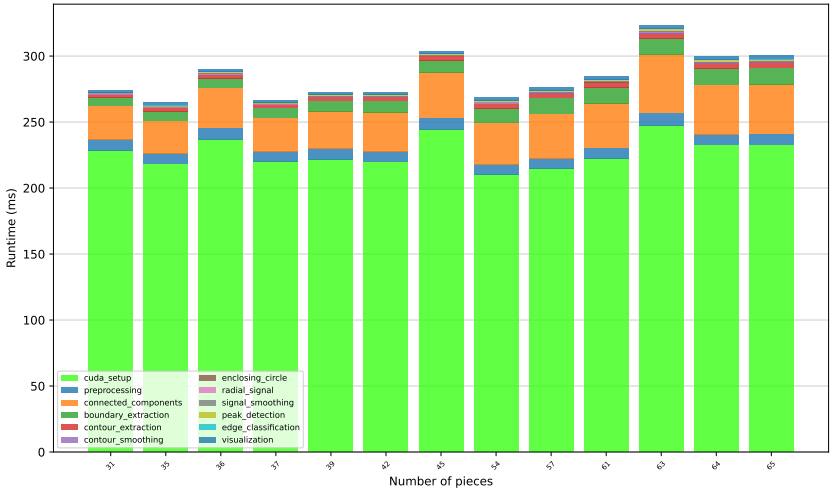

Piece count alone does not explain the runtime variation. Connected-component labeling depends primarily on the topology and complexity of the segmented foreground.

# CUDA Runtime Scaling with Image Resolution



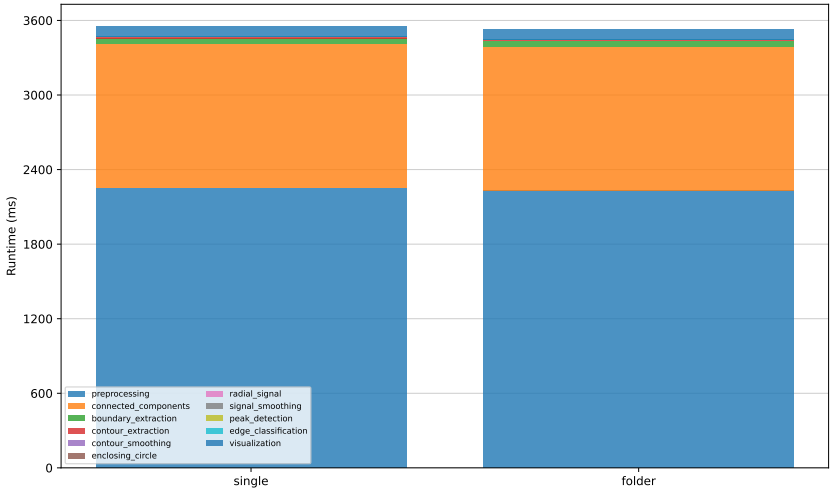

CUDA setup dominates single-image execution and grows only moderately with resolution. Connected-component labeling becomes the largest resolution-dependent processing stage.

# CUDA Runtime Across Puzzle Images




Runtime does not increase monotonically with the number of pieces. Most variation originates from connected-component labeling.

# Serial Runtime Composition by Execution Mode




Serial single-image and folder execution have nearly identical runtime compositions because no expensive process-wide GPU initialization is amortized.

# CUDA Runtime Composition by Execution Mode

Reusing the CUDA context reduces median setup time from approximately 224 ms to 22 ms per image. Connected-component labeling becomes the dominant processing stage in folder mode.

# Why Does Connected-Component Runtime Vary?

## Piece Count Is Not Sufficient

The algorithm processes the complete binary foreground topology, not only the final number of detected regions.

- Total foreground area
- Irregular component boundaries
- Narrow connections
- Local label equivalences
- Spatial arrangement of components

## Serial Implementation

Queue-based flood fill traverses every foreground component and shows large runtime differences between images.

## CUDA Implementation

Block-based union–find greatly reduces the absolute cost, but complex foreground structures still require more block unions and parent-tree resolution.

# Why Are Some CUDA Stages Slower?

## Limited Parallel Work

- Small per-piece data sets
- Short contour and signal arrays
- Branch-heavy decisions
- Irregular memory access

## Fixed GPU Overhead

- Kernel launch latency
- Device synchronization
- Host-device transfers
- Temporary-buffer management

## Hybrid Design Decision

Moore contour tracing remains CPU-side, peak filtering is only partially parallelized, and final edge classification runs on the CPU. For these small workloads, avoiding another GPU round trip is faster.

# Single-Image Crossover Point

| Resolution | Serial [ms] | CUDA [ms] | Speedup |
|---|---|---|---|
| $1024 \times 1409$ | 108.6 | 179.5 | $0.61\times$ |
| $2048 \times 2817$ | 449.0 | 212.5 | $2.11\times$ |
| $4096 \times 5635$ | 2159.1 | 263.4 | $8.20\times$ |
| $5100 \times 7016$ | 3499.8 | 290.3 | $12.06\times$ |

## Interpretation

The crossover occurs between $1024 \times 1409$ and $2048 \times 2817$. Small inputs do not provide enough work to amortize CUDA initialization, allocation, and launch overhead.

# Benchmark Limitations

- The benchmark records wall-clock stage times rather than isolated kernel, H2D, and D2H timings.
- Piece count does not represent foreground or contour complexity.
- The current CSV files do not store the exact CPU model, GPU model, CUDA version, or compiler version.
- Results describe this pipeline and data set and should not be interpreted as universal CUDA speedups.

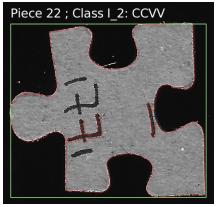
## Useful Additional Measurements

- Total foreground area and number of label equivalences
- Total contour length and number of contour samples
- Separate initialization, allocation, transfer, and kernel timings
- Hardware and software metadata stored with every benchmark run

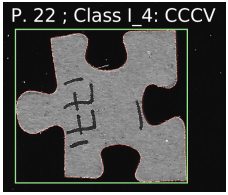
# Remaining Classification Limitations

Case 1: improved after tuning

Older

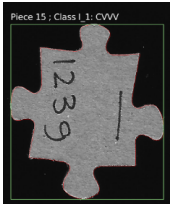



New

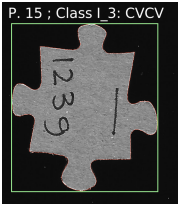



Case 3: still wrong class

Older



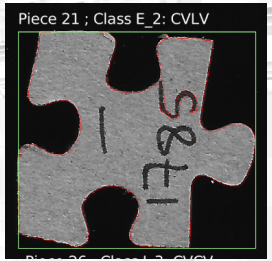

New



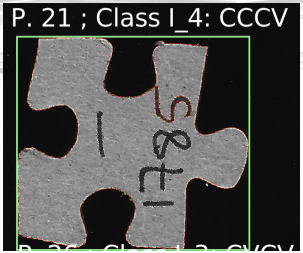

Case 1 shows that the edge-classification error could be reduced. Case 3 shows a remaining failure where the contour is reasonable, but the final edge class is still wrong.

# Remaining Classification Limitations: Printed Label

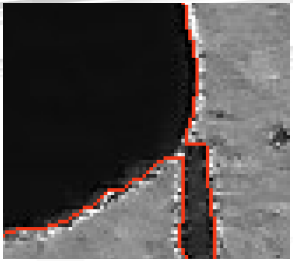
Older result




New result




Detail



In this case the final class is correct, but the printed 5 is still interpreted as part of the piece contour. The detail crop shows why this is difficult for the contour extraction step.